Serilog enables capturing this level of information very simply, preserving an object's structure when it is logged. To see how powerful this can be, add another logging call to the application. Amend the `OnGetPropertyDetails` handler in the City.cshtml.cs file as follows. Note that the placeholder name is prefixed with your old friend the @ symbol, known in Serilog as the *destructuring operator*.

Listing 12.22 The @ destructuring operator preserving object structure

```
public async Task<PartialViewResult> OnGetPropertyDetails(int id)
{
    var property = await _propertyService.FindAsync(id);
    _logger.LogInformation("Property {@property}
    ➥ retrieved by {user}", property, User.Identity.Name);      ❶
    var model = new BookingInputModel { Property = property };
    return Partial("_PropertyDetailsPartial", model);
}
```

❶ Log details of the property that was retrieved and who it was retrieved by.

This time, the rendered log message (the relevant part of which is shown in the following listing) includes a destructured version of the property variable.

Listing 12.23 The rendered log including full details of the property

```
"Level": "Information",
"MessageTemplate": "Property {@property} retrieved by {user}",
"Properties": {
  "property": {
    "_typeTag": "Property",
    "Id": 38,
    "Name": "Hotel Colonial",
    "Address": "Via Laietana",
    "CityId": 2,
    "City": null,
    "MaxNumberOfGuests": 3,
    "DayRate": 88.0,
    "SmokingPermitted": false,
    "AvailableFrom": "0001-01-01T00:00:00.0000000",
    "Deleted": null,
    "CreatorId": null,
    "Creator": null
```